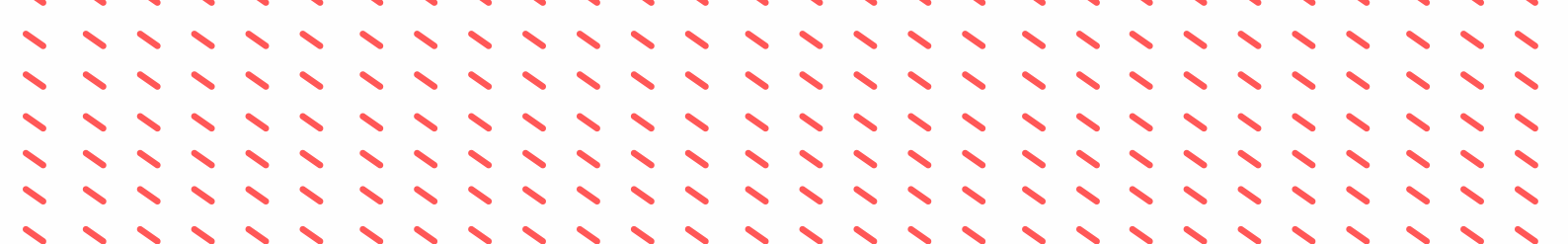




Manual Técnico Golite

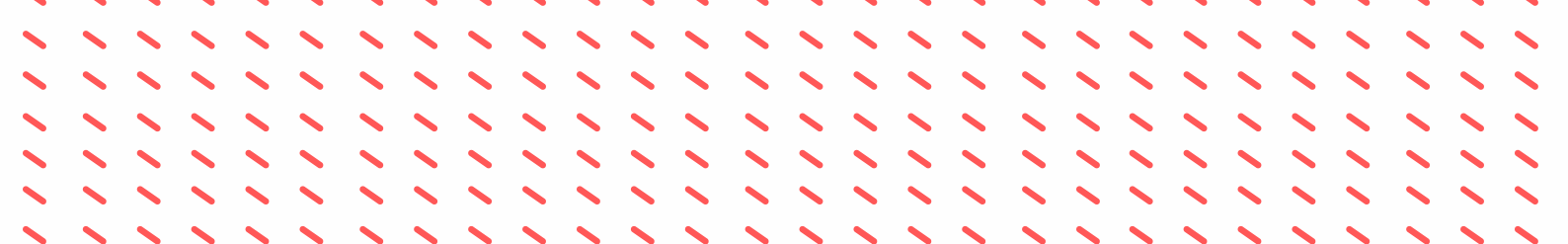
CARLOS DÍAZ - 202400245



Introducción

GoLite IDE es un entorno de desarrollo integrado diseñado para el lenguaje de programación GoLite, un subconjunto del lenguaje Go creado con fines educativos para la comprensión de los fundamentos de los compiladores e intérpretes.

Este IDE permite a los usuarios escribir, analizar y ejecutar código en lenguaje GoLite, proporcionando herramientas visuales para el análisis léxico, sintáctico y semántico del código fuente. GoLite IDE ha sido desarrollado como parte del curso de Organización de Lenguajes y Compiladores 1, con el objetivo de aplicar los conceptos teóricos de construcción de compiladores mediante el uso de herramientas como JFlex (generación de analizador léxico) y CUP (generación de analizador sintáctico) en el lenguaje de programación Java.



Descripción de la Interfaz

La interfaz se divide en tres secciones principales orientadas verticalmente para facilitar el flujo de trabajo:

1. Editor de Texto

Área destinada a la escritura directa de código o la visualización del contenido de un archivo cargado.

2. Panel de Control

Contiene los botones de acción principal:

- **Abrir:** Carga un archivo .glt
- **Guardar:** Almacena el código en un archivo .glt
- **Guardar Como:** guardar por primera vez archivos .glt
- **Ejecutar:** Inicia el análisis léxico del código

3. Consola de Salida (Inferior)

Muestra:

- **Tokens:** Resultado del análisis léxico
- **Errores:** Fallos léxicos y Sintácticos encontrados durante el proceso
- **Resultados:** Salida de la ejecución del programa

MainWindow.java

1. accionEjecutar() - Procesamiento completo del código fuente
Función: Coordina las tres fases del análisis del código GoLite.

Proceso:

- 1.Limpia los reportes anteriores (tokens, errores léxicos y sintácticos)
- 2.Análisis léxico completo - Escanea todo el código y llena la tabla de tokens
- 3.Análisis sintáctico - Crea un nuevo lexer (sin registro) y ejecuta el parser CUP
- 4.Ejecución - Si no hay errores, construye el AST y lo interpreta
- 5.Muestra resultados - Reporta conteo de tokens y errores en la consola

2. accionVerTokens() - Generación de reporte de tokens

Función: Muestra en la consola la lista completa de tokens reconocidos durante el último análisis.

Proceso:

- 1.Verifica que existan tokens en la lista estática GoliteLexer.listaTokens
- 2.Limpia la consola de salida
- 3.Imprime una tabla formateada con: número, lexema, tipo de token, línea y columna
- 4.Muestra el total de tokens encontrados

MainWindow.java

3. accionGuardar() / guardarEn() - Persistencia de archivos

Función: Gestiona el guardado de archivos fuente con extensión .glt.

Proceso de guardarEn():

1. Recibe un objeto File con la ruta destino
2. Escribe el contenido del editor en el archivo usando UTF-8
3. Actualiza archivoActual con la referencia al archivo
4. Cambia el título de la ventana para mostrar el nombre del archivo
5. Registra la operación en la consola de salida

```
14 public class MainWindow extends JFrame {
18
19     // Archivo actualmente abierto (null = nuevo sin guardar)
20     private File archivoActual = null;
21
22     public MainWindow() {
23         initComponents();
24         setupLayout();
25     }
26
27     // -----
28     // INICIALIZACIÓN
29     // -----
30     private void initComponents() {
31         setTitle(title: "GoLite IDE - Fase 1");
32         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
33         setSize(width: 1000, height: 700);
34         setLocationRelativeTo(c: null);
35
36         editorPanel = new EditorPanel();
37         consolePanel = new ConsolePanel();
38     }
39
40     private void setupLayout() {
41         setLayout(new BorderLayout());
42
43         // — Barra de herramientas —
44         JToolBar toolBar = new JToolBar();
45         toolBar.setFloatable(b: false);
```

Jflex

Método token()

Cada vez que se reconoce un patrón válido, se llama a token(tipo, valor) que:

1. Calcula la línea y columna actual
2. Si registrar es true, guarda el token en listaTokens
3. Retorna un objeto Symbol para el parser

```
private Symbol token(int tipo, Object valor) {
    int lin = yyline + 1;
    int col = yycolumn + 1;
    if (registrar) {
        listaTokens.add(new TokenReporte(yytext(), SymUtils.nombreTipo(tipo), lin, col));
    }
    return new Symbol(tipo, lin, col, valor);
}
```

Manejo de errores léxicos

Cuando se encuentra un carácter no reconocido por ninguna regla, se ejecuta la regla [^] que llama a errorLexico():

```
private void errorLexico() {
    String msg = "El símbolo \"" + yytext() + "\" no es aceptado en el lenguaje.";
    if (registrar) {
        listaErrores.add(new ErrorLexico(msg, lin, col));
        Errores.agregarLexico(msg, lin, col);
    }
}
```

Tokens Reconocidos

Categoría	Ejemplos	Símbolos en <code>sym</code>
Palabras clave	<code>var</code> , <code>if</code> , <code>else</code> , <code>for</code> , <code>func</code>	<code>VARIABLE</code> , <code>SI</code> , <code>PARA</code>
Tipos de datos	<code>int</code> , <code>float64</code> , <code>string</code> , <code>bool</code>	<code>TIPO_ENTERO</code> , <code>TIPO_FLOTANTE</code>
Operadores	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , <code>==</code> , <code>!=</code>	<code>MAS</code> , <code>MENOS</code> , <code>IGUAL</code>
Funciones embebidas	<code>fmt.Println</code> , <code>strconv.Atoi</code>	<code>IMPRIMIR</code> , <code>A_ENTERO</code>
Literales	<code>123</code> , <code>45.6</code> , <code>"hola"</code> , <code>'a'</code>	<code>LITERAL_ENTERO</code> , <code>LITERAL_CADENA</code>
Identificadores	<code>miVariable</code> , <code>contador</code>	<code>IDENTIFICADOR</code>
Delimitadores	<code>(</code> , <code>)</code> , <code>{</code> , <code>}</code> , <code>;</code> , <code>,</code>	<code>PAR_IZQ</code> , <code>LLAVE_DER</code>
Comentarios	<code>// línea</code> , <code>/* bloque */</code>	Ignorados

Integracion Con el Parser

El lexer implementa la interfaz `java_cup.runtime.Scanner`, que exige el método `next_token()`; El parser llama repetidamente a este método para obtener el siguiente token hasta recibir `sym.EOF`

```
@Override public java_cup.runtime.Symbol next_token() throws java.io.IOException
{
    int zzInput;
    int zzAction;

    // cached fields:
    int zzCurrentPosL;
    int zzMarkedPosL;
    int zzEndReadL = zzEndRead;
    char[] zzBufferL = zzBuffer;
```

Cup

El parser es generado automáticamente por la herramienta CUP a partir del archivo golite.cup. CUP toma las definiciones gramaticales y genera una clase Java (parser.java) que implementa un analizador sintáctico LALR.

Manejo de errores

```
public void syntax_error(Symbol s) {
    String msg = "Token inesperado -> '" + s.value + "'";
    ErrorSintactico err = new ErrorSintactico(msg, s.left, s.right);
    erroresSintacticos.add(err);
    Errores.agregarSintactico(msg, s.left, s.right);
}

public void unrecovered_syntax_error(Symbol s) throws Exception {
    String msg = "Error irrecuperable -> '" + s.value + "'";
    // ... registro del error
}
```

- `syntax_error()`: Se llama cuando se encuentra un token inesperado
- `unrecovered_syntax_error()`: Se llama cuando el parser no puede recuperarse del error
- Ambos registran el error en listas estáticas para reportes posteriores

Terminales y No terminales

```
terminal          VARIABLE, SI, SINO, PARA;
terminal Integer  LITERAL_ENTERO;
terminal Double   LITERAL_FLOTANTE;
terminal String   LITERAL_CADENA;
terminal String   IDENTIFICADOR;
```

```
non terminal ArrayList  program;
non terminal Nodo       expr;
non terminal NodoBloque bloque;
non terminal ArrayList  stmt_list;
```

Precedencia de Operadores (entre mas abajo mas prioridad)

```
precedence right  ASIGNAR, MAS_ASIGNAR, MENOS_ASIGNAR;
precedence left   0;
precedence left   Y;
precedence left   IGUAL, DIFERENTE;
precedence left   MENOR, MENOR_IGUAL, MAYOR, MAYOR_IGUAL;
precedence left   MAS, MENOS;
precedence left   POR, ENTRE, MODULO;
precedence right  MENOS_UNARIO;
precedence right  NO;
```

Construcción del Arbol AST

Cada regla incluye acciones que crean nodos del AST:

```
expr ::= expr:i MAS expr:d
      {: RESULT = new NodoBinario(i, "+", d, ileft, iright); :}
```

Nodos principales del AST

Nodo	Representa
NodoBinario	Operaciones con dos operandos (+ , - , * , / , == , < , etc.)
NodoUnario	Operaciones con un operando (! , - unario)
NodoLiteral	Valores constantes (enteros, flotantes, cadenas, booleanos)
NodoIdentificador	Referencia a una variable
NodoDeclVar	Declaración de variables (var x int = 10)
NodoAsignacion	Asignación de valores (x = 5)
NodoIf	Sentencia condicional if-else

Nodos principales del AST

Nodofor	Bucle <code>for</code>
NodoBloque	Bloque de código entre <code>{}</code>
NodoPrintln	Llamada a <code>fmt.Println</code>
NodoIncDec	Incremento/decremento (<code>++</code> , <code>--</code>)

Clase Abstracta Nodo

La clase `Nodo` es la clase base abstracta de la que heredan todos los nodos del Árbol de Sintaxis Abstracta (AST). Define la estructura común y el comportamiento que deben implementar todas las construcciones del lenguaje GoLite.

Propósito	Descripción
Unificar	Todas las construcciones del lenguaje (variables, if, for, operaciones, etc.) son tratadas como <code>Nodo</code>
Ubicación	Almacena línea y columna para reportes de error precisos
Ejecución	Define el método <code>ejecutar()</code> que cada subclase debe implementar
Polimorfismo	Permite recorrer el AST ejecutando cualquier nodo sin conocer su tipo específico

Entorno.java

La clase Entorno es responsable de almacenar y gestionar las variables durante la ejecución del programa GoLite, manejando correctamente los ámbitos anidados (scopes) y las señales de control de flujo (break, continue, return).

Propósito	Descripción
Almacenar variables	Mantiene un mapa de nombres de variables con su tipo y valor
Gestionar ámbitos	Soporta scopes anidados mediante referencia al entorno padre
Control de flujo	Define señales para break, continue y return
Validación semántica	Verifica que las variables existan antes de usarlas

```
src > main > java > com > mycompany > golite > Entorno.java > Entorno > ReturnSignal > ReturnSignal(Object)
1 package com.mycompany.golite;
2
3 import java.util.HashMap;
4 import java.util.Map;
5
6 /**
7  * Maneja los scopes de variables durante la ejecución.
8  */
9 public class Entorno {
10
11     //
12     // SEÑALES DE CONTROL DE FLUJO
13
14     //
15
16     /** Clase base de todas las señales */
17     public static abstract class Senal {}
18
19     /** Señal lanzada por NodoBreak */
20     public static class BreakSignal extends Senal {}
21
22     /** Señal lanzada por NodoContinue */
23     public static class ContinueSignal extends Senal {}
24
25     /** Señal lanzada por NodoReturn */
26     public static class ReturnSignal extends Senal {
27         public final Object valor;
28         public ReturnSignal(Object valor) { this.valor = valor; }
29     }
30 }
```

Errores.java

La clase Errores es un gestor centralizado que recolecta y clasifica todos los errores generados durante las diferentes fases del análisis del código GoLite.

Propósito	Descripción
Centralizar	Unificar todos los errores en un solo lugar
Clasificar	Separar errores por tipo: léxicos, sintácticos y semánticos
Reportar	Proporcionar listas completas para los reportes del IDE
Limpiar	Reiniciar el estado entre ejecuciones

```
private static List<ErrorLexico> erroresLexico = new ArrayList<>();
private static List<ErrorLexico> erroresSintactico = new ArrayList<>();
private static List<ErrorLexico> erroresSematico = new ArrayList<>();

/**
 * Agrega un error léxico
 */
public static void agregarLexico(String mensaje, int linea, int columna) {
    erroresLexico.add(new ErrorLexico(mensaje, linea, columna));
}

/**
 * Agrega un error sintáctico
 */
public static void agregarSintactico(String mensaje, int linea, int columna) {
    erroresSintactico.add(new ErrorLexico(mensaje, linea, columna));
}

/**
 * Agrega un error semántico
 */
public static void agregarSematico(String mensaje, int linea, int columna) {
    erroresSematico.add(new ErrorLexico(mensaje, linea, columna));
}
```